

Flight Delay Prediction

Two-Stage Machine Learning Engine

Sibi Ravi

1 Introduction

Flight delays are a problem leading to frustration for all humans. It has a negative effect not just on passengers, but also on those involved in the aviation business. These delays are often caused by a varying multitude of reasons. To address this issue, a two-stage predictive machine learning engine is designed to predict whether a flight will be delayed in terms of arrival, and by how many minutes the arrival is delayed. In this engine, a delay is classified as a flight arriving 15 minutes or later than its scheduled arrival time at the destination. The model contains two stages. The first stage is *classification*, where it is predicted whether a flight will be delayed or not; the flights predicted to be delayed are then passed to the second stage. The second stage is *regression*, where—taking the flights predicted to have a delay—the number of minutes the flight will be delayed is predicted. Before combining both stages, each was developed independently. The engine was trained using a dataset containing flight and weather data from 15 different airports during the years 2016 and 2017, with specific variables (features) being used. Before the multi-stage model was created, independent classification and regression stages were built, producing multiple models in order to find out which model in each stage performed best while observing the entire dataset of flights.

2 Dataset

Originally two separate datasets were provided: one containing flight information and the other containing weather data. The flight data contained information on all flights from every month of 2016 and 2017 across many different airports in the US, spanning many variables. Of those variables, 17 were selected for use (refer to Table 3). Of the airports available, only 15 were used (refer to Table 1). As a result, data points (flights) whose origin or destination was not one of the selected airports were filtered out, and unused columns from all flights were also removed. The data was separated into sets for every month, and all of these sets were then merged into one whole dataset. After merging, any data points containing blank or null values for a selected column were removed entirely.

As for the weather dataset, data was available only for the 15 selected airports, so no filtering was needed in that regard. Column filtering was performed to keep only the selected variables (refer to Table 2). Two specific variables selected have noted importance from the flight data, with these being `ArrDel15` and `ArrDelayMinutes`. The `ArrDel15` is a binary variable that will be the target during classification, where it will be predicted whether the flight will be delayed or not. The

ArrDelayMinutes is a numerical variable that will be the target during regression, where it will be predicted how many minutes is the arrival of a flight delayed by. The weather dataset contains information on the weather at each airport for every hour of the day. After filtering both the flight and weather data, the two datasets were merged based on time, date, and airport. The time in the flight data was exact while the weather data was hourly, so the flight times were rounded to the nearest hour in order to align both datasets during the merge. After all the filtering and merging, there were exactly **1,851,436** data points. In other words, there are **1,851,436** flights taken.

Table 1: Codes of the airports selected

ATL	CLT	DEN	DFW	EWR
IAH	JFK	LAS	LAX	MCO
MIA	ORD	PHX	SEA	SFO

Table 2: Weather data variables used

WindSpeedKmph	WindDirDegree	WeatherCode	precipMM
Visibility	Pressure	Cloudcover	DewPointF
WindGustKmph	tempF	WindChillF	Humidity
date	time	airport	

Table 3: Flight data variables used

FlightDate	Quarter	Year	Month
DayOfMonth	DepTime	DepDel15	CRSDepTime
DepDelayMinutes	OriginAirportID	DestAirportID	ArrTime
CRSArrTime	ArrDel15	ArrDelayMinutes	Origin
	(classification target)	(regression target)	
Dest			

2.1 Feature Selection

After merging the flight and weather datasets, extra preprocessing was conducted. Before proceeding, it was necessary to examine the relationship between the different columns (features). To do this, a correlation heatmap (refer to Figure 1) was created to view the relationships statistically between features using the correlation coefficient (r). The heatmap showed that many features had a perfect correlation ($r = 1.000$), meaning those two features would be effectively duplicates; in other cases, while not perfect, the correlation was still incredibly strong. To deal with this, one of each pair of highly correlated variables was removed. Also if there were any extremely strong, but not perfect correlations ($r > 0.90$) between two columns, one of the columns would be removed.

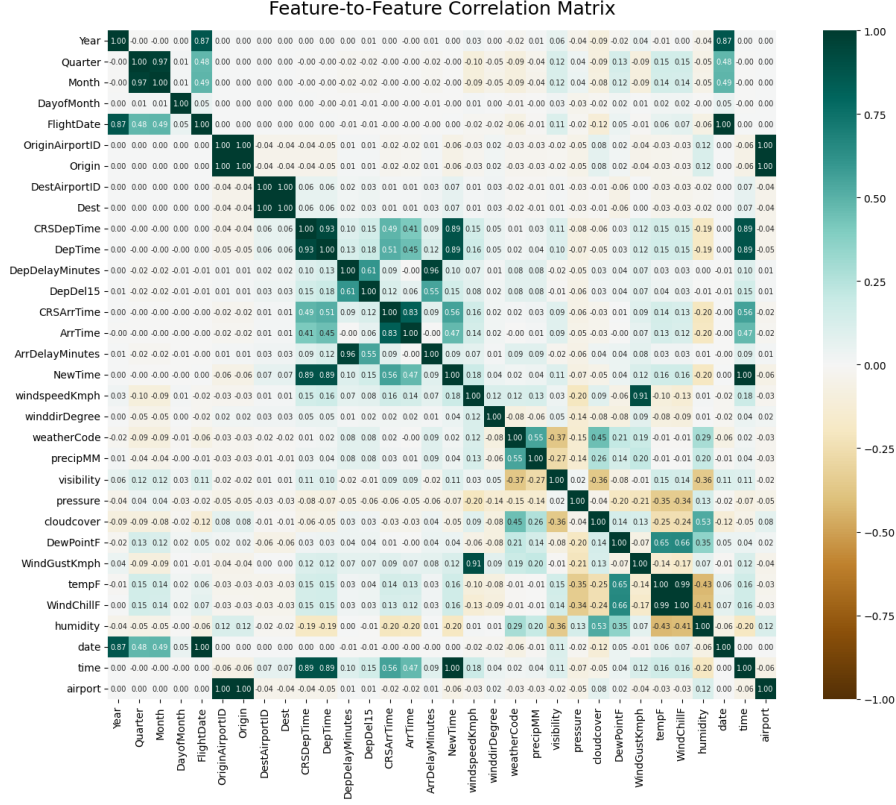


Figure 1: Feature-to-feature correlation heatmap

After removing columns due to strong correlations, additional columns were removed on the basis that it reveals information about a flight’s arrival. This machine learning engine targets whether a flight is delayed or not, so any column related to arrival would leak data about the delay. The final set of features that were used can be seen in Table 4.

Table 4: Final Features selected

WindSpeedKmph	WindDirDegree	WeatherCode	precipMM
Visibility	Pressure	Cloudcover	DewPointF
tempF	Humidity	Year	Month
DepDelayMinutes	Origin	Dest	
ArrDel15	ArrDelayMinutes		
(classification	(regression		
target)	target)		

3 Classification

With the dataset now fully cleaned, the next step was to find the best classification model. The objective of the classification models is to classify whether a flight is delayed (1.0) or not

(0.0). Before training any models, the class balance was observed between delayed and non-delayed flights. The classes were shown to be heavily imbalanced in favor of flights not being delayed (refer to Figure 2). A heavy class imbalance can lead to bias in the model.

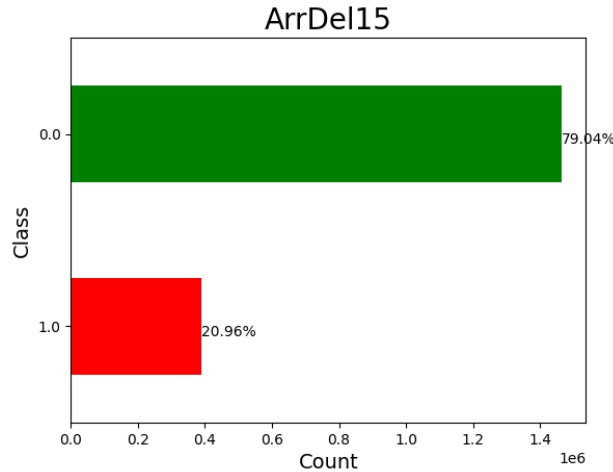


Figure 2: Class imbalance (0.0 = Not Delayed, 1.0 = Delayed)

To deal with the class imbalance, the dataset needs to be resampled. Resampling involves either undersampling or oversampling. Undersampling involves deleting data points until an equal class split is achieved, while oversampling involves adding data points until an equal class split is achieved. For this engine, an oversampling technique was used: the Synthetic Minority Over-sampling Technique (SMOTE). SMOTE balances an imbalanced dataset by generating synthetic examples of the minority class based on its nearest neighbors, instead of simply copying existing ones.

The order in which the splitting and resampling are carried out matters. A single train/test split was performed *first*, with 80% of the data randomly assigned as training data and the remaining 20% as test data. This split was stratified on **ArrDel15** so that the original proportion of delayed to non-delayed flights was preserved in both the training and test sets. Only after the split was SMOTE applied, and it was applied to the training set alone. The test set was deliberately left untouched so that it contains only real observations: resampling before splitting would place artificially generated points in the test set and inflate the reported metrics, since the test set exists solely to evaluate the trained model on genuine data.

Both target variables were carried through this same split together – **ArrDel15** as the classification target and **ArrDelayMinutes** as the regression target – so that the identical split could serve the regression stage as well (see Section 4). For the classification stage, the resampled training features are paired with the **ArrDel15** label, while the untouched test features are paired with their real **ArrDel15** values.

After splitting and resampling, the next step was creating the classification models. The classifiers used were Logistic Regression, Decision Tree, XGBoost, Random Forest, and Extra Trees. A confusion matrix (refer to Figure 3) was created for each model first, to observe the number of correct and incorrect predictions, as the matrix gives the number of true positives, false positives, true negatives, and false negatives.

- **True Positive (TP):** Flights predicted as delayed that are actually delayed.
- **False Positive (FP):** Flights predicted as delayed that are not actually delayed.
- **True Negative (TN):** Flights predicted as not delayed that are not actually delayed.
- **False Negative (FN):** Flights predicted as not delayed that are actually delayed.

		Predicted	
		Not Delayed	Delayed
Actual	Not Delayed	TN	FP
	Delayed	FN	TP

Figure 3: Confusion matrix

The metrics then used to judge the performance of each classifier were:

- **Accuracy:** measures the number of correct predictions out of all predictions.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Precision:** measures the proportion of positive predictions that are correct.

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall:** measures the proportion of actual positives correctly identified.

$$\text{Recall} = \frac{TP}{TP + FN}$$

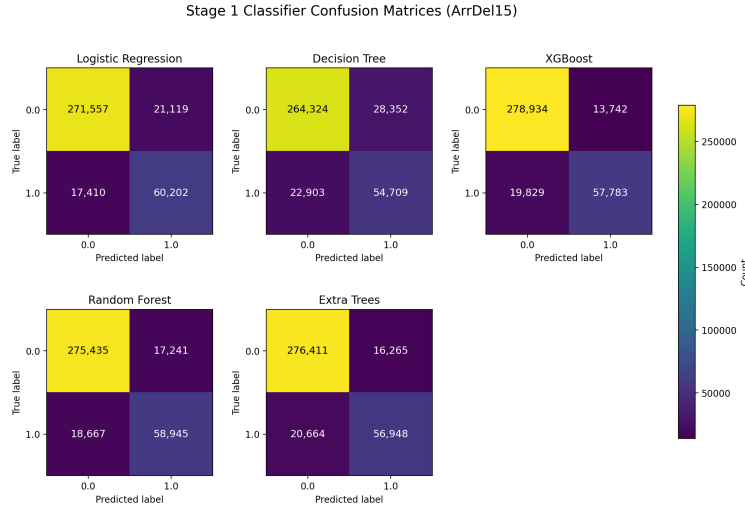
- **F1 Score:** the harmonic mean of precision and recall.

$$\text{F1 Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

- **Area Under the ROC Curve (ROC AUC):** an Area Under the Curve score is also used to judge model performance. The higher the AUC score, the better the model is at distinguishing between the different classes (refer to Table 5).

Table 5: Interpretation of AUC scores

Score	Interpretation
AUC = 0	The model is perfectly wrong—it consistently ranks delayed flights as less likely to be delayed than on-time ones.
AUC = 0.5	Equivalent to random guessing (a coin flip). The model cannot tell positives from negatives any better than chance.
$0.5 < \text{AUC} < 1.0$	The model has genuine ability to tell the two classes apart, improving as the score nears 1.0.
AUC = 1.0	A perfect classifier—it ranks every positive above every negative. Some threshold separates the classes flawlessly.

**Figure 4:** Confusion matrices for the five Stage 1 classifiers on the ArrDel15 target.**Table 6:** Classification model results

Model	Precision		Recall		F1 Score		Accuracy	AUC
	0.0	1.0	0.0	1.0	0.0	1.0		
Logistic Regression	0.94	0.74	0.93	0.78	0.93	0.76	0.90	0.903
Decision Tree	0.92	0.66	0.90	0.70	0.91	0.68	0.86	0.805
XGBoost	0.93	0.81	0.95	0.74	0.94	0.77	0.91	0.915
Random Forest	0.94	0.77	0.94	0.76	0.94	0.77	0.90	0.919
Extra Trees	0.93	0.78	0.94	0.73	0.94	0.76	0.90	0.908

Once each classification model was trained and tested, it was analyzed which classifier performed best. All computed metrics were used to decide this, with extra emphasis on F1 Score (as it is derived from both precision and recall) and AUC. When examining the results, the Random Forest

classifier had the best performance. While XGBoost matched the F1 score of Random Forest and had a miniscule higher accuracy, the latter had a slightly higher AUC score, which suggests it is more accurate at distinguishing between the different classes (delayed and not delayed). Thus, the chosen best performing classifier is Random Forest.

4 Regression

With the best classifier found, the next step was finding the best regression model. The aim of the regression model in this context is to predict the number of minutes a flight will be delayed in terms of arrival. Rather than creating a separate train/test split, the regression stage reuses the exact same split produced in the classification stage. The only difference is the target variable: the resampled training features are now paired with `ArrDelayMinutes` instead of `ArrDel15`, and the models are evaluated on the same untouched test set. Reusing a single split in this way keeps both stages aligned on identical test observations, and because SMOTE was applied only to the training set, the regression test metrics are likewise free of any synthetic data.

At this stage the regression is treated holistically: the models are trained across all flights in the resampled training set rather than only the delayed ones. The narrower focus on delayed flights alone is introduced later, in the pipeline (see Section 5).

The regression models tested were Linear Regression, Decision Tree, XGBoost, Random Forest, and Extra Trees. To evaluate the performance of each regression model, three different metrics were used:

- **Root Mean Squared Error (RMSE):**

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

- **Mean Absolute Error (MAE):**

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- **R-squared (R^2):**

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

where:

- n = number of observations
- y_i = actual value
- $\hat{y}_i$ = predicted value
- $\bar{y}$ = mean of the actual values

Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE) give an estimate of how far, on average, the predicted values are from the actual values in terms of minutes. R^2 measures the model's ability to explain the variability in the data.

Table 7: Regression model results

Regressor	RMSE	MAE	R^2
LinearRegression	11.110	6.812	0.928
DecisionTreeRegressor	15.149	7.453	0.865
XGBRegressor	13.007	6.796	0.901
RandomForestRegressor	10.728	6.085	0.932
ExtraTreesRegressor	10.767	6.043	0.932

To examine which model performs best, it must be looked for which regressor has the lowest RMSE and MAE, while R^2 needs to be as high as possible. Taking this into account and examining the metrics (refer to Table 7), the clear two best-performing regressors in each of the respective metrics are the Random Forest and Extra Trees Regressors. Both have roughly identical R^2 values, while Random Forest has a lower RMSE value, and Extra Trees has a lower MAE value. The gap between the MAE values is minimally larger than the RMSE values, so the chosen best regressor in the end, by an incredibly small margin, is the Extra Trees regressor. Extra Trees regressor has an RMSE and MAE of 10.767 and 6.043 respectively. In the context of this dataset, this means the predicted minutes of delay is off by on average 10.767 (RMSE) or 6.043 (MAE) minutes from the actual minutes of delay. The regressor has an R^2 value of 0.932, which means 93.2% of the variability in the arrival delay minutes can be explained by the model.

5 Pipeline

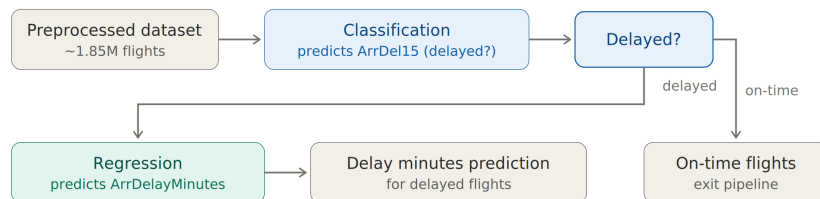


Figure 5: Pipeline flow chart

With the best classifier and regressor for this dataset found, everything was combined into one multi-stage pipeline engine. The first step of this pipeline is to take the fully preprocessed dataset ready for classification, where it will be classified if the flight is delayed (1.0) or not (0.0). First completed was the splitting of the data between training and testing. After that, in order to deal with the imbalances in the dataset, SMOTE was applied onto the training set. Once again, SMOTE was not applied on the test set, as the whole point of a test set is for testing the model that was

trained, and should contain actual data points rather than artificially generated ones. In this engine, a flight is classified as delayed when it arrives at its destination late by 15 minutes or more. The classification model used was the Random Forest classifier. The performance of the models were measured using precision, recall, F1-score, accuracy, and AUC score (refer to Table 8).

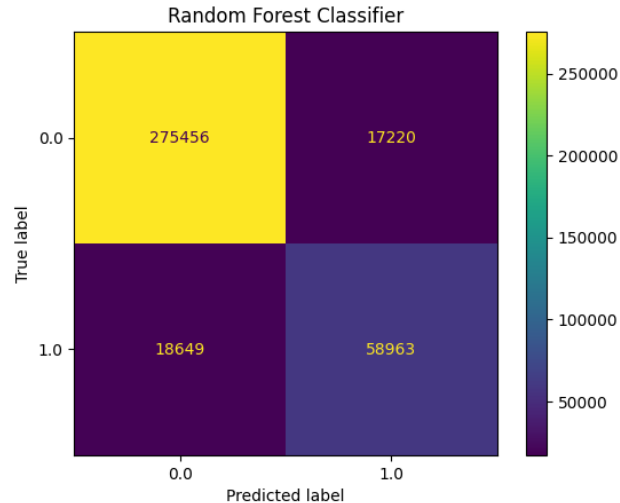


Figure 6: Classification Confusion Matrix

Table 8: Pipeline classification model results

Model	Precision		Recall		F1 Score		Accuracy	AUC
	0.0	1.0	0.0	1.0	0.0	1.0		
Random Forest	0.94	0.77	0.94	0.76	0.94	0.77	0.90	0.920

After predicting whether a flight is delayed (1.0) or not (0.0), the next step is predicting the number of arrival delay minutes for a flight that is delayed. In this pipeline regression, there will be major difference compared to the independent regression previously done, in that the focus will be on only the delayed flights rather than looking at the data from a holistic standpoint. The actually delayed flights in the pre-SMOTE training set are taken to train the Extra Trees Regressor, and the model is tested using the test data in which the classification model predicted a delay. The RMSE, MAE, and R^2 values were calculated in order to measure the performance (refer to Table 9). The RMSE comes to 19.898, which means the predicted number of delay minutes is on average 19.898 minutes from the actual. The MAE comes to 15.208, which means the predicted number of delay minutes is on average 15.208 minutes from the actual. The R^2 comes to 0.929, which means 92.9% of the variability in the predicted arrival delay minutes can be explained by the model.

Table 9: Pipeline regression model results

Regressor	RMSE	MAE	R^2
ExtraTreesRegressor	19.898	15.208	0.929

5.1 Bin Analysis

Once the number of delay minutes was predicted, the flights were grouped into bins based on the range of minutes delayed (15–200 minutes, 200–400, 400–600, 600–800, 800–1000, and 1000+). After separating the flights into bins, the performance of the trained Extra Trees regression model was tested within each bin, with the metrics (RMSE, MAE, R^2) given as output (refer to Table 10).

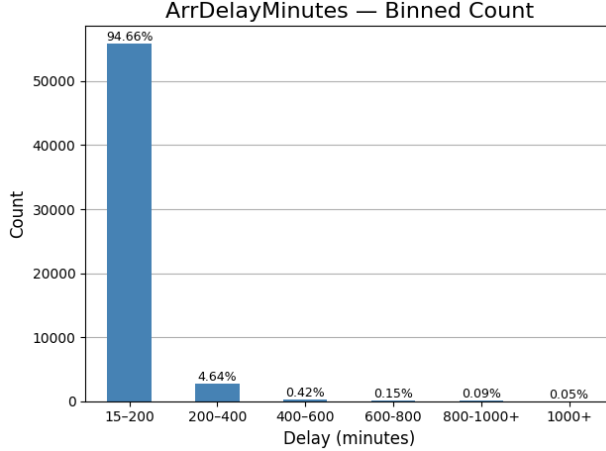


Figure 7: Bin count for ArrDelayMinutes of delayed flights

Table 10: Bin-by-bin regression model results

Delay Range (min)	n	RMSE	MAE	R^2
15–200	55,812	17.10	12.38	0.831
200–400	2,735	29.97	19.99	0.659
400–600	245	31.46	19.46	0.688
600–800	87	28.85	18.51	0.746
800–1000	54	18.98	13.58	0.878
1000+	30	25.08	19.51	0.987

As seen in Figure 7, the overwhelming majority (94.66%) of delayed flights fall within the 15–200 minute range. This bin is therefore the one that matters most, and it is precisely where the model performs best: its RMSE of 17.10 and MAE of 12.38 are both *lower* than the full pipeline regression figures of 19.898 and 15.208 (refer to Table 9). In other words, on the flights that make up nearly all of the delayed population, the model is more accurate than the aggregate metric suggests. The higher pipeline RMSE is driven by the small number of flights in the extreme-delay bins (the 200–400 through 600–800 bins all sit near 30 RMSE); because RMSE squares each error, these few large misses inflate the overall figure out of proportion to their count.

The R^2 values require more careful interpretation. Across the bins, R^2 is consistently lower than for the full delayed dataset (with the exception of the 1000+ bin). This is a range-restriction artifact rather than a sign of poor fit: each bin covers a narrow range of delay minutes, so the variance in the actual values is small, and R^2 falls even when the absolute error is reasonable. The 1000+

bin spans a far wider range, restoring the variability and pushing its R^2 back up. For this reason, per-bin performance is best judged on RMSE and MAE, with R^2 reported only for completeness. The upper bins should also be read with caution: with only 245, 87, 54, and 30 flights respectively, their metrics are estimated from very few observations and are correspondingly noisy.

6 Conclusion

A two-stage machine learning engine has now been fully created. It first predicts whether a flight is delayed using Random Forest, and then takes the flights predicted to be delayed and uses Extra Trees to predict how many minutes each is delayed. Splitting the problem in this way suits the data well, since the large majority of flights are not delayed and the delay minutes are heavily concentrated at the lower end.

The performance metrics for both stages show the engine performs well. The classifier reaches an F1 Score of 0.77 for delayed flights and an AUC of 0.920, while the regressor achieves an RMSE of 19.898, an MAE of 15.208, and an R^2 of 0.929. The aggregate regression error is also somewhat inflated by a small number of extreme delays: on the 15–200 minute range, which covers 94.66% of delayed flights, the model is in fact more accurate than these overall figures suggest.

There remains room for improvement. The reliance on `DepDelayMinutes` (see Section 7) and the limited number of flights in the extreme-delay bins are the main limitations, and future work could address these as well as explore additional features to better capture the causes of delay.

7 Leakage Concern

One concern for potential error, or rather potential leakage, comes with using the `DepDelayMinutes` as a feature. The `DepDelayMinutes` contains the minutes of delay for a flight that departed late. One may bring the concern that there is a leakage here as when a flight is delayed in terms of departure, it often means the arrival will also be delayed. However, `DepDelayMinutes` is kept because it can capture non-weather delay causes such as mechanical problems or runway congestion that the weather features alone cannot. It should be noted that not all delays are caused by weather factors.